

Cómo funciona el Monitor

La ETL explicada desde cero: de los datos a la web

Qué vas a encontrar

Lee en orden la primera vez; después úsala como consulta.

Apartado		Para qué te sirve
1	La idea en una página	Entender el sistema completo de un vistazo
2	Vocabulario mínimo	Las palabras que salen todo el rato: repositorio, commit, push, workflow...
3	Las tres fuentes de datos	Qué se extrae, cómo se transforma y dónde se carga cada una
4	GitHub Actions	Qué es, qué se ejecuta cada día y a qué hora
5	El archivo YML, línea a línea	Leer y tocar el «manual de instrucciones» del robot
6	Qué es YAML	Las reglas del formato y por qué falló ayer
7	El script del promedio	Qué hace R/promedio.R por dentro
8	De los datos a la web	GitHub Pages, la página y la caché
9	Tu día a día	Comprobar, lanzar a mano, subir cambios, qué hacer si algo falla
10	Mapa de archivos y pendientes	Dónde está cada cosa y qué falta por hacer

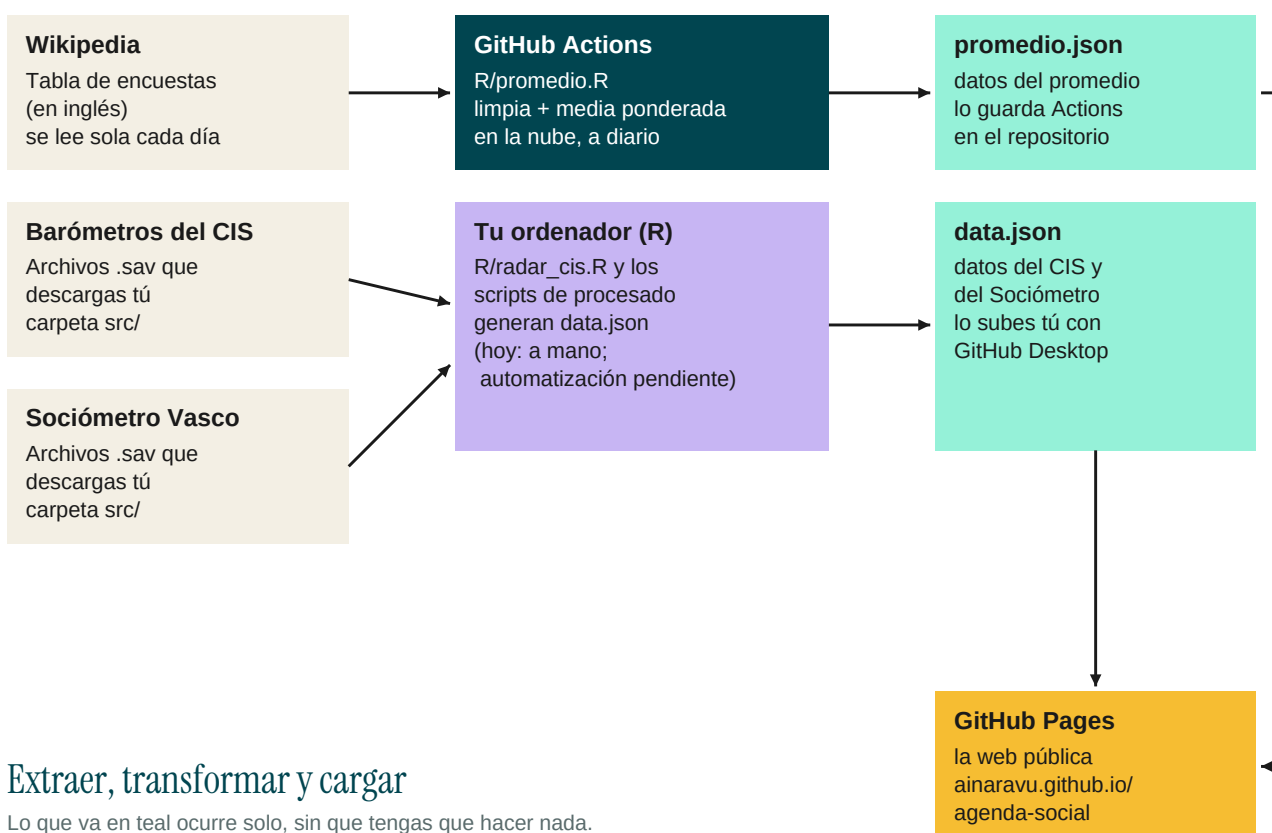
Esta guía describe el sistema tal y como quedó el 6 de octubre de 2026. Si se cambia algo importante (horarios, scripts, fuentes), conviene actualizar la sección correspondiente.

1 · La idea en una página

Tres fuentes, un robot y una web.

Una **ETL** es el camino que siguen los datos desde donde nacen hasta donde se usan. Son tres pasos:

Paso	Qué significa	En el Monitor
E · Extraer	Ir a buscar los datos a su origen	Leer la tabla de encuestas de Wikipedia; descargar los .sav del CIS y del Sociómetro
T · Transformar	Limpiarlos y calcular lo que necesitas	Scripts de R: limpiar sondeos, media ponderada, porcentajes por grupo, radares...
C · Cargar	Dejarlos donde los va a leer la aplicación	Dos archivos de datos (promedio.json y data.json) que la web lee en cada visita



Extraer, transformar y cargar

Lo que va en teal ocurre solo, sin que tengas que hacer nada.

La clave del diseño: **la web y los datos están separados**. La página (index.html y app.js) solo dibuja; los números viven en dos archivos JSON. Actualizar la web es, sencillamente, sustituir esos archivos en GitHub.

Lo automático. El promedio de encuestas: cada mañana un «robot» de GitHub lee Wikipedia, recalcula y publica. No necesitas tener el ordenador encendido.

Lo manual (de momento). El CIS y el Sociómetro: los .sav los descargas tú y los datos se suben desde tu carpeta con GitHub Desktop. Está previsto automatizarlo con una tarea programada en tu ordenador.

2 · Vocabulario mínimo

Las diez palabras que vas a oír.

Palabra	Qué es, en llano	En este proyecto
Git	Programa que guarda versiones de una carpeta: quién cambió qué y cuándo. Permite volver atrás.	Está detrás de GitHub y de GitHub Desktop; no lo usas directamente.
GitHub	Web donde se guardan carpetas con Git («repositorios») en la nube.	Tu cuenta: AinaraVU.
Repositorio (repo)	Una carpeta de proyecto con todo su historial.	AinaraVU/agenda-social. Es público: cualquiera puede verlo (solo hay datos agregados).
Commit	Una «foto» guardada de los cambios, con un mensaje que explica qué cambia.	Ej.: «Radar de problemas», «Promedio de encuestas 2026-10-06».
Push	Subir tus commits de tu ordenador a GitHub.	Botón «Push origin» de GitHub Desktop.
Pull / Fetch	Fetch: mirar si hay cambios nuevos en GitHub. Pull: bajarlos a tu carpeta.	Necesario porque el robot hace commits del promedio cada día.
GitHub Desktop	Aplicación con botones para hacer commit, push y pull sin escribir comandos.	Lo que usas para subir los cambios.
GitHub Actions	El «robot» de GitHub: ejecuta tareas en sus ordenadores cuando ocurre algo o a una hora.	Calcula el promedio a diario y publica la web.
Workflow	Una receta de Actions: cuándo se lanza y qué pasos hace. Se escribe en un archivo .yaml.	.github/workflows/publicar.yaml
YAML / .yaml	Formato de texto para escribir configuraciones de forma legible.	El formato del workflow (sección 6).
Cron	Forma estándar de escribir horarios repetidos: minuto, hora, día, mes, día de la semana.	«23 4 * * *» = todos los días a las 04:23 UTC.
Runner	El ordenador temporal (Ubuntu) que GitHub presta para ejecutar el workflow. Se borra al acabar.	Instala R, ejecuta el script y desaparece.
GitHub Pages	Servicio gratuito de GitHub que publica una web estática desde un repositorio.	ainaravu.github.io/agenda-social
JSON	Formato de texto para guardar datos (listas, nombres y valores). Lo leen las webs.	promedio.json y data.json
Scraping	Leer de forma automática datos publicados en una web.	Leer la tabla de encuestas de Wikipedia.
API	Puerta de entrada «para programas» de un servicio web, más estable que la página visible.	Usamos la API de Wikipedia (MediaWiki) para pedir la página.
UTC	Hora universal. GitHub programa en UTC. Madrid es UTC+2 en verano y UTC+1 en invierno.	04:23 UTC = 06:23 en verano, 05:23 en invierno.

3 · Las tres fuentes de datos

Cada una sigue su propia ETL.

A · Promedio de encuestas (automático, diario)

	Detalle
Extraer	Página de Wikipedia «Opinion polling for the 2026 Spanish general election», sección <i>Voting intention estimates</i> . Se pide a la API de Wikipedia con un identificador propio (User-Agent), como pide Wikipedia. Hay una tabla por año (2026, 2025, 2024 y 2023).
Transformar	Se limpian los sondeos (fecha del trabajo de campo, empresa, muestra y % de cada partido) y se calcula una media ponderada diaria desde el 23-07-2023 hasta hoy (detalle en la sección 7).
Cargar	Se escribe <code>promedio.json</code> , se guarda en el repositorio con un commit automático y se publica en la web.
Dónde se ve	Pestaña «Evolución encuestas».
Quién lo hace	GitHub Actions, solo, cada día (sección 4).

B · Barómetro del CIS (manual por ahora)

	Detalle
Extraer	Descargas los microdatos (.sav, formato SPSS) de cada barómetro desde la web del CIS y los guardas en la carpeta <code>src/barometro_cis/</code> . Hay 34, de septiembre de 2023 a septiembre de 2026.
Transformar	Scripts de R en tu ordenador leen los .sav (paquete haven), ponderan con PESO y calculan porcentajes por grupo. <code>R/radar_cis.R</code> calcula los radares (problemas, voto + simpatía e intención de voto por grupo).
Cargar	Todo va a <code>data.json</code> , que se sube a GitHub con GitHub Desktop. Al subirlo, el workflow publica la web.
Dónde se ve	Pestaña «Barómetro del CIS».

C · Sociómetro Vasco (manual por ahora)

Igual que el CIS: los .sav (16 oleadas, de octubre de 2021 a junio de 2026) están en `src/sociometro_vasco/` y sus resultados van también a `data.json`. Se ve en la pestaña «Sociómetro Vasco».

¿Por qué el CIS y el Sociómetro no se hacen en la nube? Porque los microdatos se descargan a mano y no deben publicarse: el repositorio es público. Solo se suben los porcentajes agregados (`data.json`), nunca los .sav.

Pendiente: automatizarlo con `R/actualizar_local.R`, que cada día miraría si hay un .sav nuevo, lo procesaría, comprobaría el resultado y lo subiría solo. Falta escribir el procesado completo (`procesar_cis.R` y `procesar_sociometro.R` son plantillas).

4 · GitHub Actions

Un robot que trabaja en los ordenadores de GitHub.

GitHub Actions es un servicio de GitHub que **ejecuta tareas automáticamente**. Tú le das una receta (el workflow) que dice *cuándo* actuar y *qué pasos* seguir. Cuando llega el momento, GitHub presta un ordenador temporal (el *runner*, con Linux Ubuntu), ejecuta los pasos y lo borra. Para repositorios públicos es gratis.

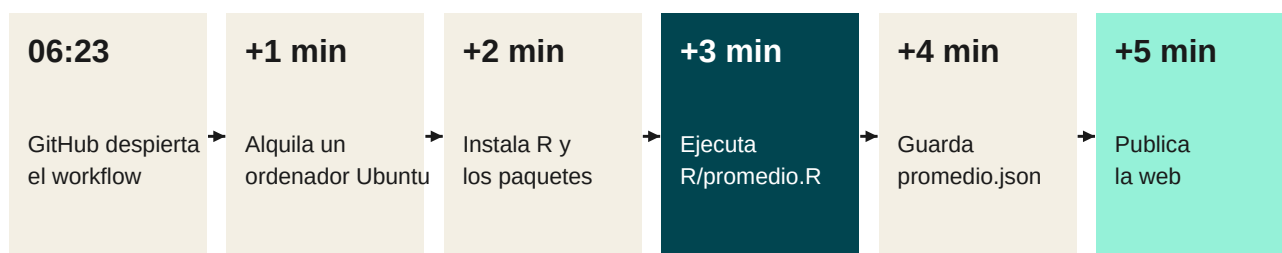
Cuándo se lanza nuestro workflow

Disparador	Cuándo ocurre	Qué hace
<code>schedule</code> (horario)	Cada día a las 04:23 UTC y, de respaldo, a las 07:41 UTC	Calcula el promedio y publica la web
<code>workflow_dispatch</code> (a mano)	Cuando pulsas «Run workflow» en la pestaña Actions	Calcula el promedio y publica la web
<code>push</code> (subida)	Cada vez que alguien sube cambios a la rama main (tú con GitHub Desktop)	Solo publica la web (no recalcula el promedio)

Programado (UTC)	Madrid en verano (CEST, UTC+2)	Madrid en invierno (CET, UTC+1)
04:23	06:23	05:23
07:41 (respaldo)	09:41	08:41

GitHub no garantiza la hora exacta: en momentos de mucha carga retrasa o incluso salta tareas programadas, sobre todo a las horas en punto. Por eso hay dos horas y ninguna es en punto. Si las dos funcionan, la segunda no cambia nada: solo guarda el promedio cuando ha cambiado.

Qué pasa cada mañana, paso a paso



Hora de Madrid en verano. A las 09:41 se repite como respaldo. Si no hay sondeos nuevos, no se cambia nada.

El workflow tiene dos «trabajos» (jobs) que van uno detrás de otro:

Job	Pasos	Resultado
promedio	1. Descarga el repositorio (checkout). 2. Instala R. 3. Instala los paquetes rvest, xml2, httr2, jsonlite, dplyr y stringr. 4. Ejecuta Rscript R/promedio.R. 5. Si promedio.json ha cambiado, hace un commit «Promedio de encuestas AAAA-MM-DD» y lo sube.	promedio.json actualizado en el repositorio
publicar	1. Descarga el repositorio ya actualizado. 2. Copia a una carpeta _site solo lo público: index.html, app.js, data.json y promedio.json. 3. Lo empaqueta y lo despliega en GitHub Pages.	La web pública actualizada
Si el job promedio falla (por ejemplo, porque Wikipedia cambia su tabla), el job publicar no se ejecuta: la web sigue mostrando el último promedio bueno y GitHub te envía un correo avisando del fallo.		

5 · El archivo YML, línea a línea

`.github/workflows/publicar.yml`

Este archivo es la receta del robot. GitHub busca automáticamente cualquier `.yml` dentro de la carpeta `.github/workflows/` del repositorio. Lo que va detrás de una almohadilla (`#`) es un comentario: GitHub lo ignora.

```
name: Promedio diario y publicación      # nombre que verás en la pestaña Actions

on:                                     # CUÁNDO se lanza
  schedule:                             # a una hora fija (cron, en UTC)
    - cron: "23 4 * * *"               # 04:23 UTC = 06:23 Madrid en verano
    - cron: "41 7 * * *"               # 07:41 UTC = 09:41 Madrid (respaldo)
  push:                                 # cuando se suben cambios...
    branches: [main]                  # ...a la rama principal
  workflow_dispatch:                   # botón «Run workflow» (a mano)

permissions:                           # QUÉ PUEDE HACER el robot
  contents: write                      # escribir en el repo (commit del promedio)
  pages: write                         # publicar en GitHub Pages
  id-token: write                     # identificarse ante Pages al desplegar

concurrency:                           # nunca dos ejecuciones a la vez
  group: publicar                     # si llega otra, espera; no cancela la actual
  cancel-in-progress: false
```

Cómo se lee un cron

Posición	1 · minuto	2 · hora	3 · día del mes	4 · mes	5 · día de la semana
«23 4 * * *»	23	4 (UTC)	* = todos	* = todos	* = todos

Ejemplos: «0 5 * * 1-5» = de lunes a viernes a las 05:00 UTC; «30 6 1 * *» = el día 1 de cada mes a las 06:30 UTC.

```

jobs:
    # QUÉ hace: dos trabajos
    promedio:
        if: github.event_name != 'push'      # en un push NO se recalcula el promedio
        runs-on: ubuntu-latest              # ordenador temporal con Linux
        steps:
            - uses: actions/checkout@v4      # 1. bajar el repositorio
            - uses: r-lib/actions/setup-r@v2  # 2. instalar R
              with:
                use-public-rspm: true        # (paquetes ya compilados: más rápido)
            - uses: r-lib/actions/setup-r-dependencies@v2
              with:
                packages: any::rvest, any::xml2, any::httr2, any::jsonlite, any::dplyr, any::stringr
            - name: Calcular promedio.json
              run: Rscript R/promedio.R      # 4. ejecutar el script
            - name: Guardar promedio.json si ha cambiado
              run: |                         # 5. varias órdenes seguidas (la | lo permite)
                git config user.name "github-actions[bot]"
                git config user.email "41898282+github-actions[bot]@users.noreply.github.com"
                git add promedio.json
                git diff --cached --quiet || git commit -m "Promedio de encuestas $(date +%F)"
                git pull --rebase            # por si alguien subió algo mientras tanto
                git push

    publicar:
        needs: promedio                    # va DESPUÉS de «promedio»...
        if: always() && (needs.promedio.result == 'success' || needs.promedio.result == 'skipped')
        # ...y solo si salió bien o se saltó (push)

        runs-on: ubuntu-latest
        environment:
            name: github-pages
            url: ${ steps.deploy.outputs.page_url }
        steps:
            - uses: actions/checkout@v4
              with: { ref: main }           # versión más reciente (con el promedio nuevo)
            - name: Preparar la web (solo lo público)
              run: |
                mkdir _site
                cp index.html app.js data.json _site/
                [ -f promedio.json ] && cp promedio.json _site/ || true
            - uses: actions/configure-pages@v5 # preparar Pages
            - uses: actions/upload-pages-artifact@v3
              with: { path: _site }          # empaquetar la carpeta _site
            - id: deploy
              uses: actions/deploy-pages@v4  # publicar

```

¿Qué es «uses»? Son piezas ya hechas por otros (acciones) que se reutilizan: [actions/checkout](#) lo mantiene GitHub y [r-lib/actions](#) lo mantiene el equipo de R. El número tras la @ es la versión. ¿Y «run»? Una orden que se escribe tal cual en la terminal del runner.

6 · Qué es YAML

Un formato donde los espacios importan.

YAML es una forma de escribir configuraciones que se lee casi como una lista en un papel. Tiene pocas reglas, pero son estrictas:

- **clave: valor.** Siempre con un espacio después de los dos puntos: «name: Promedio», no «name:Promedio».
- **La sangría marca la jerarquía.** Lo que está «dentro» de algo va más a la derecha, con espacios (aquí, de dos en dos).
- **Nunca tabuladores.** Solo espacios. Un tabulador invisible rompe el archivo.
- **Las listas empiezan por guion:** «- cron: ...» es un elemento de una lista.
- **«#» empieza un comentario** hasta el final de la línea.
- **Entre comillas** los textos con caracteres especiales (como los asteriscos del cron).

El error del 6 de octubre

Al cambiar los horarios se perdió la línea «on:» y la sangría de debajo. Así quedaba (mal) y así debía quedar (bien):

Mal

```
name: Promedio diario y publicación
```

```
schedule:
```

```
- cron: "23 4 * * *"
```

```
push:
```

```
  branches: [main]
```

```
workflow_dispatch:
```

x falta «on:» encima
x sangría de 5 espacios
x queda «suelto»

Bien

```
name: Promedio diario y publicación
```

```
on:
```

```
  schedule:
```

```
    - cron: "23 4 * * *" ✓ 2 espacios
```

```
    - cron: "41 7 * * *" ✓ 4 espacios
```

```
  push:
```

```
    branches: [main]
```

```
  workflow_dispatch:
```

Sin «on:», GitHub no sabe qué es «schedule» y rechaza el archivo. El editor de GitHub marca estos errores en rojo antes de guardar: si no ves nada en rojo, el formato es válido.

Truco. Para editar el .yaml desde github.com: abre el archivo, pulsa el lápiz, cambia solo lo necesario y pulsa «Commit changes». Guardar el archivo cuenta como un push, así que la web se vuelve a publicar sola.

7 · El script del promedio

Qué hace R/promedio.R por dentro.

1 · Descargar

Pide la página a la API de Wikipedia (con hasta 4 reintentos si falla la conexión) y localiza las tablas de la sección *Voting intention estimates*; el año de cada tabla sale del título que la precede. Para pruebas puede leer una copia guardada de la página con la variable `PROMEDIO_HTML`.

2 · Limpiar cada sondeo

- **Empresa y medio:** «GAD3/ABC» se separa en empresa (GAD3) y medio (ABC).
- **Fecha:** el trabajo de campo («29 sep–1 oct») se convierte en fecha de inicio y de fin, y se usa el **punto medio**. Si cruza el año, se ajusta.
- **Muestra:** «1,000» → 1000. Si falta, se usa la mediana de los demás.
- **% de cada partido:** el primer número de la celda; los escaños que van detrás se ignoran.
- **Celdas compartidas:** el CIS comparte fecha y muestra con sus reestimaciones; se respetan esas celdas combinadas.
- **Fuera:** filas de elecciones, sondeos que solo dan escaños y duplicados (el mismo sondeo publicado por dos medios).
- **Podemos:** en 2023 iba dentro de Sumar; hasta su primer sondeo propio no tiene valor (no cuenta como 0).

3 · Calcular la media ponderada

Para cada día desde el 23-07-2023 hasta hoy se toman los sondeos de los últimos 60 días y se ponderan:

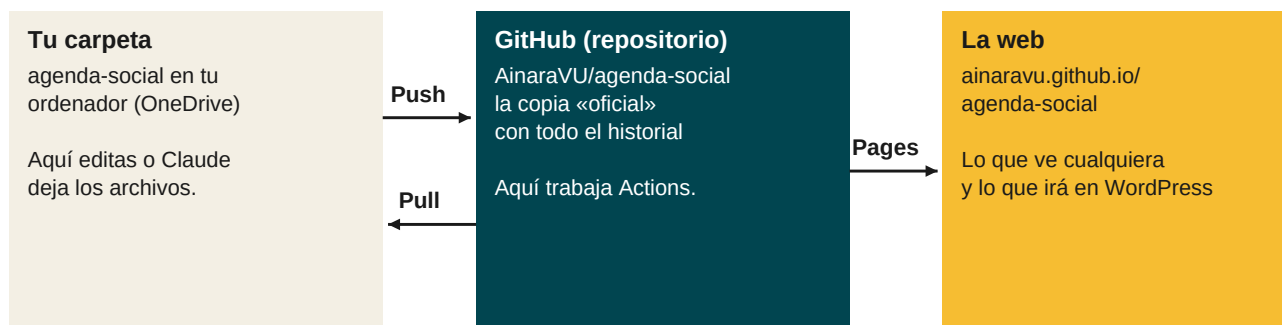
Factor	Regla	Por qué
Antigüedad	Un sondeo de hace 14 días pesa la mitad que uno de hoy	Lo reciente informa más
Tamaño de la muestra	$\sqrt{(\text{muestra} / 1000)}$, con un tope de 3.000	Más entrevistas, más fiable; el tope evita que una encuesta domine
Empresa	El más reciente de cada empresa pesa 1; el anterior, 0,5; el siguiente, 0,25...	Que las que publican cada semana no pesen más
Mínimo	Hacen falta al menos 3 sondeos en la ventana	Evitar valores con muy poca información
Suavizado	Media gaussiana de unos 7 días sobre la serie diaria	Quitar los escalones que produce cada sondeo nuevo

4 · Comprobar y guardar

El script **no sobrescribe** promedio.json, y termina con error (aviso por correo), si falta alguno de los partidos del gráfico, si algún valor sale fuera de 0–60 %, si PP + PSOE + Vox del último día queda fuera de 50–95 % o si hay más de 5 sondeos menos que en la versión anterior (señal de que la lectura falló a medias). Si todo está bien, escribe `promedio.json` con la fecha de actualización, la fuente y su licencia (CC BY-SA 4.0), el método, los partidos y sus colores, la tendencia diaria, todos los sondeos y el último valor de cada partido.

8 · De los datos a la web

GitHub Pages, la página y la caché.



Push: subes tus cambios a GitHub. Pull: te bajas los cambios que hizo GitHub (p. ej. el promedio diario).

Pages: GitHub convierte el repositorio en la web pública (lo hace el propio workflow).

GitHub Pages

Pages publica una web **estática**: archivos que el navegador descarga tal cual, sin servidor ni base de datos. Por eso es gratis y muy estable. Nuestro workflow solo publica lo necesario (index.html, app.js y los dos JSON); los scripts de R y la documentación están en el repositorio, pero no en la web.

Qué pasa cuando alguien abre la web

- El navegador descarga **index.html** (la estructura y los estilos).
- index.html pide **data.json** y **promedio.json** pidiendo siempre la versión más reciente. Si promedio.json fallara, el resto funciona igual.
- Después carga **app.js**, que dibuja los gráficos con la librería ECharts.
- Todo ocurre en el navegador de quien visita: GitHub solo entrega los archivos.

¿No ves los cambios? El navegador guarda copias para ir más rápido (caché). Recarga con **Ctrl + F5** (en Mac, Cmd + Shift + R). Tras un push, Pages tarda 1–2 minutos en publicar.

En WordPress (pendiente)

La web de S&M mostrará el Monitor dentro de una página con un bloque «HTML personalizado» que contiene un *iframe* (una ventana a otra web):

```
<iframe src="https://ainaravu.github.io/agenda-social/" title="Monitor de la opinión pública" style="width:100%;height:90vh;border:0;display:block" loading="lazy"></iframe>
```

Como el iframe apunta a Pages, WordPress no hay que tocarlo nunca más: se actualiza solo.

9 · Tu día a día

Comprobar, lanzar, subir y arreglar.

Comprobar que el promedio se ha actualizado

- En github.com/AinaraVU/agenda-social, abre la pestaña **Actions**.
- Cada fila es una ejecución. ✓ **verde** = bien; ✗ **rojo** = falló; círculo amarillo = en marcha.
- Las del horario muestran «schedule»; las de un push, el mensaje del commit.
- En la pestaña **Code** verás commits «Promedio de encuestas AAAA-MM-DD» del robot (solo los días con sondeos nuevos).
- En la web, el pie del gráfico dice «Actualizado el ...».

Lanzarlo a mano

Actions → «Promedio diario y publicación» (columna izquierda) → **Run workflow** → **Run workflow**.
Tarda unos 3–5 minutos.

Subir cambios desde tu carpeta (GitHub Desktop)

Paso	Botón	Por qué
1	Fetch origin y, si aparece, Pull origin	Bajar antes lo que hizo el robot (el promedio diario); si no, el push puede fallar
2	Revisar la lista de archivos y desmarcar lo que no quieras subir	Ej.: capturas o archivos de prueba
3	Escribir un resumen y pulsar Commit to main	El mensaje ayuda a saber después qué cambió
4	Push origin	Sube a GitHub; la web se publica sola en 1–2 minutos

Si algo falla

Síntoma	Qué significa	Qué hacer
Correo de GitHub «Run failed»	Una ejecución acabó en rojo. La web sigue con lo anterior.	Abrir la ejecución en Actions, ver el paso en rojo y pasar el texto del error a Claude
Falla «Calcular promedio.json»	Normalmente Wikipedia cambió la tabla o una comprobación saltó	Revisar el mensaje «No se guarda promedio.json: ...»; ajustar el script
Hoy no hay ejecución «schedule»	GitHub retrasó o saltó la tarea	Lanzarla a mano; la de respaldo de las 09:41 debería cubrirlo
El push de GitHub Desktop da error	GitHub tiene commits que tú no tienes	Fetch → Pull y volver a hacer Push
Aviso «workflow desactivado»	GitHub desactiva los horarios tras 60 días sin actividad en el repo	Actions → el workflow → «Enable workflow»

Nunca subas los .sav ni la carpeta src/ al repositorio: es público. El archivo .gitignore ya excluye las carpetas datos/ y logs/.

10 · Mapa de archivos y pendientes

Dónde está cada cosa.

Archivo	Qué es	Quién lo cambia
index.html	La página: estructura, estilos y carga de los datos	Tú / Claude, cuando cambia el diseño
app.js	Los gráficos (ECharts) y las pestañas	Tú / Claude, cuando cambia un gráfico
data.json	Datos agregados del CIS y del Sociómetro	Tus scripts de R (hoy a mano)
promedio.json	Datos del promedio de encuestas	El robot de GitHub Actions, a diario
.github/workflows/publicar.yml	La receta del robot (horarios y pasos)	Solo si cambian horarios o pasos
R/promedio.R	Scraping de Wikipedia + media ponderada	Si cambia Wikipedia o el método
R/radar_cis.R	Calcula los radares del CIS desde los .sav	Al añadir barómetros
R/actualizar_local.R	Revisión diaria de archivos nuevos en tu ordenador	Pendiente de poner en marcha
R/comprobar_data_json.R	Verifica que un data.json nuevo tiene la misma forma	Lo usa actualizar_local.R
R/procesar_cis.R	Procesado completo del CIS (plantilla)	Pendiente de escribir
R/procesar_sociometro.R	Procesado completo del Sociómetro (plantilla)	Pendiente de escribir
docs/	Documentación técnica detallada (01 a 08)	Se actualiza con cada cambio
.gitignore	Lista de lo que nunca se sube (datos, logs/...)	Rara vez

Pendientes

Tarea	Para qué
1 Escribir el procesado completo del CIS y del Sociómetro en R	Regenerar data.json desde los .sav sin intervención
2 Programar la tarea diaria en tu ordenador (taskscheduleR)	Que un .sav nuevo aparezca en la web el mismo día
3 Incrustar el Monitor en WordPress (iframe)	Que se vea en la web de S&M;
4 (Opcional) Dirección propia, p. ej. panel.tudominio.com	Una URL de marca en lugar de github.io

Recuerda: lo que está en GitHub manda. Tu carpeta es una copia de trabajo; GitHub es la versión oficial y la web sale de ahí.